

find 정리

범위 안에서 원하는 값을 선형 검색하는 알고리즘입니다.

사용처 (Where)

- vector/list/deque에서 특정 값 찾기
- 값이 존재하는지 확인
- 찾은 위치를 기반으로 삭제/수정

방법 (How to Use)

- begin/end 범위와 찾을 값을 넘김
- 찾으면 해당 iterator 반환
- 못 찾으면 end 반환
- set/map은 멤버 함수 find가 더 적합

기본 정보

- 필요 헤더: `#include <algorithm>`
- 기본 선언: `auto it = find(v.begin(), v.end(), x);`

왜 써야 할까? (Why)

- 직접 반복문보다 의도가 분명함
- 컨테이너 종류와 상관없이 범위 기반으로 사용
- 찾은 위치를 다른 알고리즘/erase와 연결 가능

주요 함수

함수/문법	의미
<code>find(first,last,x)</code>	x 검색
반환값	찾은 위치 iterator
실패	last 반환
<code>find_if</code>	조건으로 검색

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {10, 20, 30};
    auto it = find(v.begin(), v.end(), 20);

    if (it != v.end()) cout << "found";
}
```

find 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
find	$O(n)$
set/map find	$O(\log n)$
unordered find	평균 $O(1)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
개발 난이도	find 함수 호출만 알면 됨	반복문과 조건을 직접 설계	qsort/직접 반복문 사용
코드 길이	짧고 의도가 분명함	길어지고 실수 지점 증가	자료형 처리와 캐스팅이 번거로움
안정성	표준 라이브러리라 검증됨	구현 실수 가능	포인터/범위 실수 가능
추천 상황	대부분의 일반 상황	원리를 공부하거나 특수 최적화	C 코드와 연동할 때

응용: 찾은 값 삭제

find로 위치를 찾은 뒤 erase에 전달하여 해당 값을 삭제합니다.

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4};
    auto it = find(v.begin(), v.end(), 3);

    if (it != v.end()) v.erase(it);

    for (int x : v) cout << x << ' ';
}
```

처음 배울 때 자주 하는 실수

- 못 찾았는데 *it를 하면 안 됨
- 정렬된 vector에서 빠른 검색은 binary_search/lower_bound 고려
- map/set에서는 std::find보다 멤버 find가 빠름

비슷한 항목과 차이

- find_if는 조건 검색
- count는 개수 세기
- binary_search는 정렬된 범위에서 빠른 존재 확인

한 줄 정리: find은(는) 'vector/list/deque에서 특정 값 찾기' 상황에서 먼저 떠올리면 좋은 STL입니다.